$$f(x) = x^2 \tag{1}$$

$$\frac{df}{dx} = \frac{f(x+\varepsilon) - f(x)}{\varepsilon} \tag{2}$$

$$= \frac{(x+\varepsilon)^2 - x^2}{\varepsilon} \tag{3}$$

$$= \frac{x^2 + 2x\varepsilon + \varepsilon^2 - x^2}{\varepsilon} \tag{4}$$

$$= \frac{\varepsilon(2x+\varepsilon)}{\varepsilon} \tag{5}$$

$$= 2x + \varepsilon \tag{6}$$

$$2x \approx 2x + \varepsilon \tag{7}$$

Here we simply take the ratio of the output interval to the input interval, both of which are infinitely small because ε is an infinitesimal number. We can algebraically reduce the expression to $2x + \varepsilon$, and since ε is infinitesimal, $2x + \varepsilon$ is infinitely close to $2x$, which we take as the true derivative of the original function $f(x) = x^2$. Remember, we're taking ratios of oriented intervals that can be positive or negative. We not only want to know how much a function stretches (or squeezes) the input, but whether it changes the direction of the interval. There is a lot of advanced mathematics justifying all of this (see nonstandard analysis or smooth infinitesimal analysis) but this process works just fine for practical purposes.

Why is differentiation a useful concept in deep learning? Well, in machine learning we are trying to *optimize* a function, which means finding the input points to the function such that the output of the function is a maximum or minimum over all possible inputs. That is, given some function, $f(x)$, we want to find an x such that $f(x)$ is smaller than any other choice of x; we generally denote this as $argmin(f(x))$. Usually we have a loss function (or cost or error function) that takes some input vector, a target vector, and a parameter vector and returns the degree of error between the predicted